

石材幕墙污渍检测系统

软件设计规格说明文档

Software Design Specification



项目人员：于广淳（2353740）

项目助理：周慧星（2351289）

同济大学软件工程专业开发团队

2026 年 6 月

目录

1. 引言.....	4
1.1 文档目的.....	4
1.2 系统范围.....	4
2. 系统概述.....	4
2.1 项目背景.....	4
2.2 系统功能概述.....	5
2.2.1 图像上传与管理功能.....	5
2.2.2 幕墙污渍检测功能.....	5
2.2.3 检测报告导出功能.....	5
2.2.4 历史记录查询功能.....	5
3. 系统架构设计.....	6
3.1 架构总体设计.....	6
3.2 技术选型.....	8
3.3 前端架构.....	8
3.3.1 前端目录结构.....	8
3.3.2 前端代理层.....	9
3.4 后端架构.....	9
3.4.1 后端目录结构.....	9
3.4.2 后端处理流程.....	10
3.5 系统集成架构.....	10
4. 接口设计.....	11
4.1 API 接口清单.....	11
4.2 创建检测任务 POST /api/detections.....	11
4.3 查询检测列表 GET /api/detections.....	13
4.4 获取检测详情 GET /api/detections/{task_id}.....	13
4.5 重试检测 POST /api/detections/{task_id}/retry.....	13
4.6 获取签名 URL GET /api/detections/{task_id}/signed-url.....	14
4.7 删除检测 DELETE /api/detections/{task_id}.....	14
4.8 批量删除 DELETE /api/detections/batch/delete.....	14
5. 数据设计.....	14
5.1 数据库设计.....	14
5.2 数据字典.....	14
5.2.1 检测任务表 (inspection_tasks).....	14
5.2.2 任务图像表 (task_images).....	15
5.2.3 检测结果表 (detection_results).....	16
5.2.4 检测区域表 (result_regions).....	16
5.3 数据流图.....	17
5.3.1 污渍检测功能数据流图.....	17
5.3.2 历史记录查询数据流图.....	18
6. 组件设计.....	19
6.1 后端组件.....	19
6.1.1 应用入口 (app/main.py).....	19

6.1.2 API 路由层 (app/api/)	19
6.1.3 核心层 (app/core/)	19
6.1.4 服务层 (app/services/)	20
6.1.5 后台任务层 (app/workers/)	20
6.2 前端组件	20
6.2.1 检测控制台 (pages/stonedirty/detection.vue)	20
6.2.2 历史记录 (pages/stonedirty/history.vue)	20
6.2.3 API 封装层 (api/detections.ts)	21
6.3 组件依赖关系	21
7. 用户界面设计	23
7.1 系统整体布局	23
7.2 检测控制台页面	23
7.2.1 左侧操作区	23
7.2.2 右侧展示区	23
7.3 历史记录页面	24
7.3.1 搜索与筛选	24
7.3.2 数据表格	24
7.3.3 详情抽屉	24
7.4 检测报告导出	26
8. 安全设计	27
8.1 身份认证	27
8.2 权限控制	27
9. 部署设计	27
9.1 后端部署	27
9.2 前端部署	28
9.3 环境变量配置	28
10. 附录	29
10.1 缩略语与术语表	29
10.2 参考文档	29
10.2.1 项目文档	30
10.2.2 技术参考	30

1. 引言

1.1 文档目的

本文档旨在为石材幕墙污渍检测系统提供完整的软件设计规格说明，涵盖系统架构设计、组件设计、数据设计、接口设计、用户界面设计、安全设计及部署设计，作为开发团队实施和维护的技术依据，同时也为项目评审与验收提供参考。

本系统基于 YOLO 系列深度学习模型与图像处理算法，采用前后端分离架构，实现石材幕墙污渍的自动检测、面积计算、结果可视化及历史记录管理等功能。系统前端集成于已有的智慧幕墙管理平台（基于 Nuxt.js 3），后端独立部署为 FastAPI 微服务。

1.2 系统范围

石材幕墙污渍检测系统的主要客户及终端用户为同济大学土木工程团队管理层、物业管理公司及建筑维护公司。

本系统作为同济大学计算机科学与技术学院智慧幕墙项目的子系统，专注于实现以下功能范围：

- 石材幕墙污渍检测模块的开发与集成（基于 YOLO 系列模型，支持本地推理与云端 API 推理双模式）

- 污渍区域检测与分类（轻度 light、中度 moderate、重度 severe 三级分类）

- 污渍占比计算与整体清洁度评估

- 前端检测控制台与历史记录页面的开发，集成于已有智慧幕墙管理平台

- 基于 MySQL 和 OSS 的数据存储与管理（4 张数据表）

- 通过 Docker 容器化技术实现后端服务部署

本系统不涉及硬件设备的采购与维护。前端作为子模块集成于已有智慧幕墙项目前端，训练所得 YOLO 系列模型可供复用。

2. 系统概述

2.1 项目背景

随着现代建筑中石材幕墙的广泛应用，其表面的污渍、粉尘和污染物问题日益突出。石材幕墙不仅具有美观的视觉效果，还发挥着隔热、隔音、抗风压等重要功能。然而，受气候变化、环境污染及建筑使用等因素影响，幕墙表面极易积累污渍，不仅影响建筑美观，还可能损害其使用寿命和功能。

目前，传统的幕墙检测主要依赖人工巡检，存在效率低下、成本高昂、高空作业安全风险大等明显缺陷。基于此，本项目利用计算机视觉与深度学习技术，开发一套智能石材幕墙污渍检测系统，实现对石材幕墙污渍的快速、精准检测，以达到以下目标：

- 提升检测效率：将人工巡检转变为自动化检测，大幅缩短检测时间

降低检测成本：减少对专业巡检人员的依赖，降低人力成本

提高安全性：避免人工高空作业，降低安全风险

提供数据支持：为建筑的长期维护和清洁计划提供科学、量化的数据依据

2.2 系统功能概述

2.2.1 图像上传与管理功能

(1) 图像上传：用户可通过系统上传建筑幕墙图片，支持拖拽上传、文件选择上传及批量上传三种方式，支持 JPEG、PNG 等主流图像格式，单文件大小不超过 50MB。系统会对超过 100KB 的图像自动进行压缩处理以加快上传速度。

(2) 图像命名与元数据录入：上传过程中弹出命名对话框，用户需为图像指定名称，并填写建筑名称、楼层、分区、描述等元数据信息，便于后续管理与检索。

(3) 待检测队列管理：成功上传的图片自动进入待检测队列列表，支持单独上传、批量上传、删除、切换推理模式（本地/云端）等操作。

(4) 检测图片选择：用户可从待检测队列中选取指定图片进行检测，也可一键批量提交所有待检测图片。

2.2.2 幕墙污渍检测功能

(1) 污渍检测与分类：系统调用训练好的 YOLO 系列模型对图像进行目标检测，直接识别图像中的污渍区域，并按严重程度分为 light（轻度）、moderate（中度）、severe（重度）三级。

(2) 双模式推理：系统支持本地模型推理和云端 API 推理两种模式。本地模式使用 Ultralytics YOLO 加载本地 best.pt 模型文件；云端模式通过 HTTP 请求调用远程推理服务。

(3) 智能同步/异步处理：图像文件大小不超过 2MB 时采用同步处理（直接返回结果），超过 2MB 时采用异步后台处理（通过 FastAPI BackgroundTasks）。

(4) 结果可视化：系统将检测结果（带标注的图像、污渍类型、污渍面积占比、整体清洁度等）返回至前端界面进行可视化展示，包括原图与检测后图片的对比展示。

2.2.3 检测报告导出功能

(1) 当次报告导出：在检测控制台页面完成单次检测后，用户可点击【导出 PDF】按钮，以 PDF 格式导出本次检测报告，内容包括任务信息、建筑信息、污渍类型、污渍占比、区域明细、原图与检测后图片等。

(2) 历史报告导出：在历史记录查询页面，用户可针对任意一条历史检测记录点击【导出 PDF】按钮，导出该条记录的检测报告。

(3) 批量报告导出：用户可勾选多条历史记录，点击【导出 PDF】按钮批量导出多条检测记录的报告。

2.2.4 历史记录查询功能

(1) 记录存储与归档：系统将每次污渍检测结果自动保存为历史记录，包含检测

时间、建筑名称、楼层、分区、图片名称、污渍类型、污渍占比、整体清洁度等信息。

（2）多维筛选：用户可按建筑名称、检测状态（待处理/处理中/已完成/失败）、时间范围对历史记录进行过滤。

（3）分页展示：历史记录按检测时间倒序排列，分页显示，每页默认 10 条记录，支持翻页浏览。

（4）详情查看：点击历史记录可打开抽屉式详情面板，查看完整检测信息，包括原图与检测后图片的对比、区域明细表格（污渍类型、置信度、边界框坐标）等。

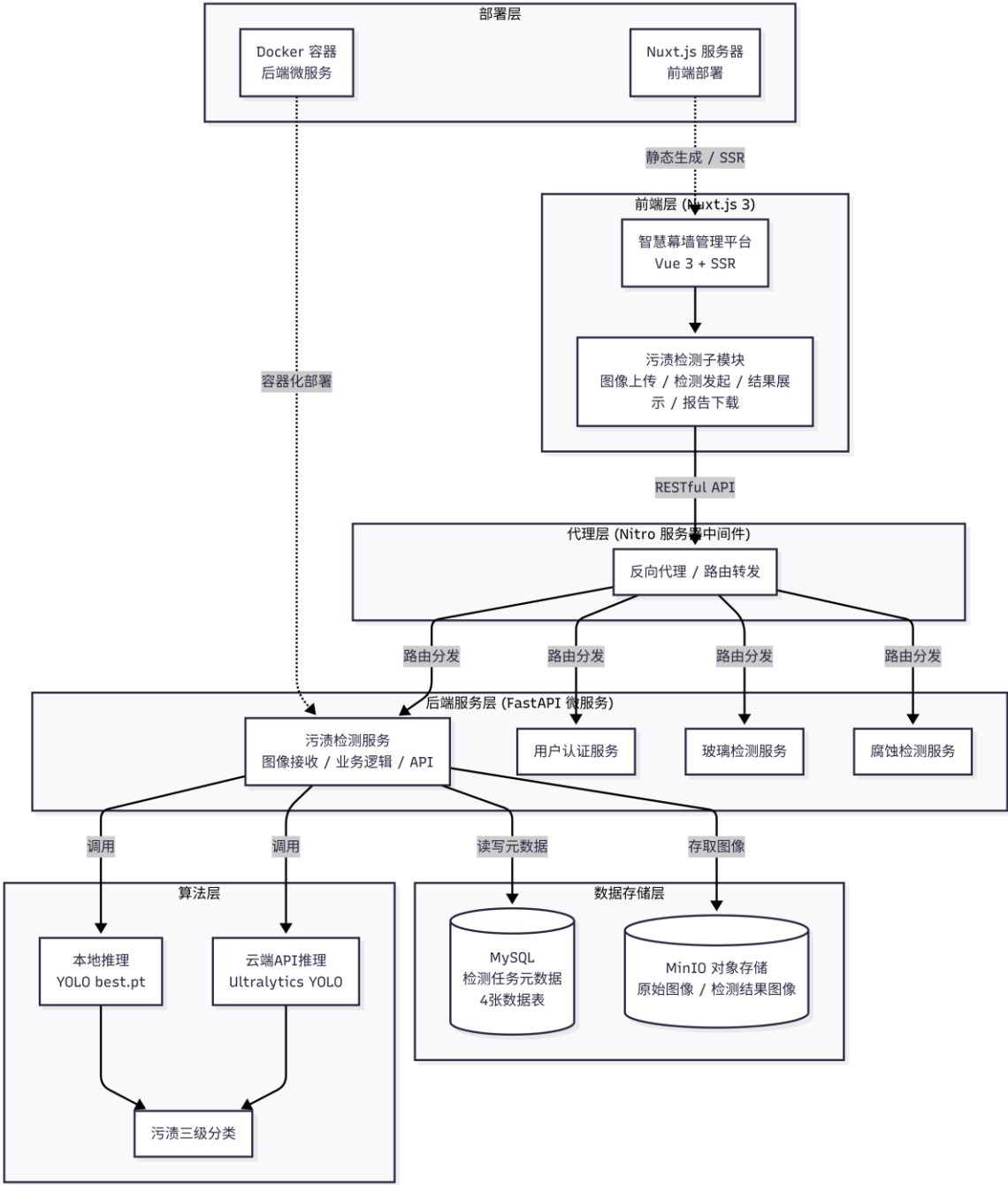
（5）重试与删除：对失败的检测任务支持一键重试；支持单条删除和批量删除历史记录。

（6）客户端缓存：历史列表采用 15 分钟客户端缓存机制，减少重复请求，提升响应速度。

3. 系统架构设计

3.1 架构总体设计

本系统采用前后端分离的微服务架构。污渍检测功能作为智慧幕墙管理平台的一个子模块，前端页面集成于已有平台前端，后端独立部署为 FastAPI 微服务。整体架构如下：



前端层：基于 Nuxt.js 3 框架（Vue 3 + SSR 可选），集成于智慧幕墙管理平台，负责图像上传、检测发起、结果展示与报告下载等用户操作，通过 RESTful API 与后端通信。

代理层：Nuxt.js Nitro 服务器中间件作为反向代理，将前端请求路由到多个后端服务（污渍检测、用户认证、玻璃检测、腐蚀检测等）。

后端服务层：基于 FastAPI 框架提供污渍检测业务逻辑处理和 API 服务，包括图像接收、YOLO 模型推理、结果处理及数据持久化等核心功能。

算法层：集成 YOLO 系列深度学习模型（Ultralytics YOLO），支持本地推理（best.pt）和云端 API 推理两种模式，实现污渍检测与三级分类。

数据存储层：采用 MySQL 数据库（4 张数据表）存储检测任务元数据与结果，采用 OSS 对象存储（MinIO 兼容）保存原始图像及检测结果图像。

部署层：后端通过 Docker 容器化技术部署，前端通过 Nuxt.js 内置服务器或静态生成方式部署。

3.2 技术选型

各层技术选型如下表所示：

层次	技术	版本/说明
前端框架	Nuxt.js 3	3.17.3，基于 Vue 3.5.13，CSR 模式（ssr: false）
UI 组件库	Element Plus + Nuxt UI Pro	提供表格、表单、对话框、抽屉等组件
CSS 框架	Tailwind CSS	通过 Nuxt UI 集成
HTTP 客户端	Axios	前后端 API 通信
图表库	ECharts 5.6.0	数据可视化
PDF 生成	jsPDF + html2canvas	客户端 PDF 报告导出
状态管理	Pinia + useState	前端状态管理
后端框架	FastAPI	0.116.1，Python 异步 Web 框架
ASGI 服务器	Uvicorn	0.35.0
数据库	MySQL 8.0+	通过 PyMySQL 直连，无 ORM
对象存储	OSS（MinIO 兼容）	图像文件存储
ML 模型	Ultralytics YOLO	本地 best.pt 模型推理
图像处理	Pillow + NumPy	图像读取、绘制标注
认证	PyJWT	HS256 Bearer Token
数据验证	Pydantic	2.11.7，请求/响应数据校验
容器化	Docker + Docker Compose	后端服务部署

3.3 前端架构

前端采用 Nuxt.js 3 框架，以 CSR（客户端渲染）模式运行。污渍检测模块的前端页面作为子模块集成于已有的智慧幕墙管理平台前端。

3.3.1 前端目录结构

污渍检测相关的核心文件：

路径	说明
pages/stonedirty/detection.vue	检测控制台页面（新版，含队列管理）

路径	说明
pages/stonedirty/history.vue	历史记录查询页面
pages/stonedirty/mainpage.vue	旧版检测页面（兼容保留）
api/detections.ts	新后端 API 封装（调用 FastAPI 服务）
api/stain.js	旧版 API 封装（调用遗留服务）
types/detection.ts	TypeScript 类型定义
composables/usePendingQueue.ts	待检测队列状态管理
utils/detectionPdfExport.ts	PDF 报告导出工具
utils/compressImage.ts	图像压缩工具
layouts/default.vue	主布局（含侧边栏导航）
middleware/auth.global.js	全局认证中间件
server/middleware/proxy.ts	Nitro 反向代理中间件

3.3.2 前端代理层

Nuxt.js Nitro 服务器中间件负责将前端请求路由到对应的后端服务：

前端路径前缀	目标后端	用途
/detection-api/*	localhost:8081	新版 FastAPI 污渍检测后端
/api/auth/*	Nitro 本地 API	登录、注册、登出、用户信息
/api/*	8.159.143.133:8000	用户账户/认证服务
/api/detect/*	8.153.161.229:8003	玻璃裂纹/平整度检测
/api/corrosion/*	Nitro 本地 API	腐蚀检测
/predict, /history	47.102.208.89:8007	遗留污渍检测服务
/oss/*	8.159.143.133:9000	OSS 对象存储

3.4 后端架构

后端采用 FastAPI 框架，独立部署于 8081 端口，提供污渍检测的完整 RESTful API。

3.4.1 后端目录结构

路径	说明
app/main.py	FastAPI 应用入口，CORS 配置，路由注册
app/api/detections.py	检测 API 路由（CRUD + 重试 + 签名 URL）

路径	说明
app/api/health.py	健康检查端点
app/core/config.py	环境变量配置 (Settings 数据类)
app/core/auth.py	JWT 认证 (PyJWT HS256 解码)
app/core/response.py	统一 JSON 响应格式 (ok/fail)
app/schemas/detection.py	Pydantic 数据模型
app/services/model_adapter.py	YOLO 模型适配器 (本地+云端推理)
app/services/mysql_repo.py	MySQL 数据仓库 (PyMySQL 直连)
app/workers/tasks.py	后台异步检测任务
sql/final_detection_schema.sql	MySQL 数据库 Schema

3.4.2 后端处理流程

- (1) 同步处理流程 (图像 ≤ 2MB) :
- 1. 接收 multipart/form-data 请求 (image 文件 + 元数据表单字段)
 - 2. 验证 JWT Token, 提取 user_id
 - 3. 验证图像格式 (仅支持 image/jpeg、image/png)
 - 4. 将原始图像上传至 OSS, 获取存储路径
 - 5. 在 MySQL 中创建 inspection_tasks、task_images、detection_results 三条记录 (状态为 pending)
 - 6. 更新任务状态为 processing
 - 7. 调用 model_adapter 执行 YOLO 推理 (本地或云端)
 - 8. 将检测后图片上传至 OSS, 更新 task_images.processed_image_path
 - 9. 更新 inspection_tasks (污渍检测结果)、detection_results (指标)、result_regions (区域明细)
 - 10. 返回完整的任务详情 JSON
- (2) 异步处理流程 (图像 > 2MB) :
- 1. 步骤 1-5 与同步流程相同
 - 2. 通过 FastAPI BackgroundTasks 将检测任务提交到后台线程
 - 3. 立即返回任务详情 (状态为 pending)
 - 4. 后台线程执行步骤 6-9
 - 5. 前端通过轮询 (每 2 秒) 查询任务状态直到完成或失败

3.5 系统集成架构

污渍检测系统作为智慧幕墙管理平台的子模块, 与其他模块共享前端框架和用户认证体系:

模块	前端路由	后端服务	说明
污渍检测	/stonedirty/*	localhost:8081 (FastAPI)	本文档描述的系统
玻璃裂纹检测	/glass-inspection/crack	8.153.161.229:8003	独立后端服务
玻璃平整度检测	/glass-inspection/flatness	8.153.161.229:8003	独立后端服务
腐蚀检测	/corrosion/*	Nitro 本地 API	Nuxt 服务器端处理
振动监测	/vibration/*	远程服务	独立后端服务
性能评估	/resilience/*	远程服务	独立后端服务
用户管理	/userManager, /accountManagement	8.159.143.133:8000	统一认证服务

4. 接口设计

本系统后端基于 FastAPI 框架，对外暴露 RESTful API 接口，所有接口均需携带 JWT Bearer Token 进行身份认证。统一响应格式为：

`{"code": 0, "data": ..., "message": "..."} (code=0 表示成功，非 0 表示失败)`

4.1 API 接口清单

方法	路径	说明	认证
GET	/api/health	健康检查	否
POST	/api/detections	创建检测任务	是
GET	/api/detections	查询检测列表	是
GET	/api/detections/{task_id}	获取检测详情	是
POST	/api/detections/{task_id}/retry	重试失败任务	是
GET	/api/detections/{task_id}/signed-url	获取图像签名 URL	是
DELETE	/api/detections/{task_id}	删除单条检测	是
DELETE	/api/detections/batch/delete	批量删除检测	是

4.2 创建检测任务 POST /api/detections

接口说明：接收用户上传的图像文件及元数据，创建检测任务并执行污渍检测。小文件（≤2MB）同步返回结果，大文件异步处理。

请求方式：POST，Content-Type: multipart/form-data

请求参数：

参数名	类型	必填	说明
image	File	是	图像文件（仅支持 image/jpeg 、 image/png）
building_name	string	是	建筑名称
location_floor	int	否	楼层号
location_section	string	否	分区名称
description	string	否	描述信息
inference_mode	string	否	推理模式：“local”（默认）或“cloud”

返回参数（data 对象）：

字段	类型	说明
id	string	任务 ID（自增主键）
userId	string	用户 ID
buildingName	string	建筑名称
locationFloor	int null	楼层
locationSection	string null	分区
status	string	任 务 状 态： pending/processing/done/failed
createdAt	datetime	创建时间
updatedAt	datetime	更新时间
imageName	string	图片名称
imagePath	string	OSS 原始图像路径
imageSignedUrl	string	原始图像签名访问 URL
processedImagePath	string null	OSS 检测后图像路径
processedImageSignedUrl	string null	检测后图像签名 URL
summary	string null	检测结果摘要
stainDetected	boolean null	是否检测到污渍
stainType	string null	主 要 污 渍 类 型： light/moderate/severe
affectedAreaPercentage	number null	污渍面积占比（%）

字段	类型	说明
regions	array	检测区域明细列表
metrics	object null	检测指标 (runtimeMs, overallCleanliness, inferenceMode)
errorMessage	string null	错误信息 (失败时)

4.3 查询检测列表 GET /api/detections

接口说明：分页查询当前用户的检测任务列表，支持多维筛选。

请求参数 (Query)：

参数名	类型	必填	说明
current_page	int	否	页码，默认 1，最小 1
size	int	否	每页条数，默认 10，范围 1-100
status	string	否	状态筛选：pending/processing/done/failed
building_name	string	否	建筑名称模糊搜索
start_time	string	否	开始时间 (ISO 格式)
end_time	string	否	结束时间 (ISO 格式)

返回参数：

字段	类型	说明
items	array	检测任务列表 (字段同 4.2, 但 regions 为空数组, imageSignedUrl 等为 null)
pagination.current_page	int	当前页码
pagination.size	int	每页条数
pagination.total	int	总记录数

4.4 获取检测详情 GET /api/detections/{task_id}

接口说明：获取指定检测任务的完整详情，包括区域明细和签名 URL。

路径参数：task_id (任务 ID)

返回参数：完整的任务详情对象 (字段同 4.2)。

4.5 重试检测 POST /api/detections/{task_id}/retry

接口说明：对失败的检测任务重新提交执行。将任务状态重置为 pending，从 OSS 重新下载图像并提交后台检测。

路径参数：task_id（任务 ID）

返回参数：{"id": task_id, "status": "pending"}

4.6 获取签名 URL GET /api/detections/{task_id}/signed-url

接口说明：获取检测任务的原始图像和检测后图像的签名访问 URL。

返回参数：

字段	类型	说明
id	string	任务 ID
imagePath	string	OSS 存储路径
imageSignedUrl	string	签名访问 URL

4.7 删除检测 DELETE /api/detections/{task_id}

接口说明：删除指定检测任务及其关联的所有数据（任务、图像记录、检测结果、区域明细）。

路径参数：task_id（任务 ID）

4.8 批量删除 DELETE /api/detections/batch/delete

接口说明：批量删除多条检测记录。

请求参数：task_ids（Query 参数，逗号分隔的任务 ID 列表）

5. 数据设计

5.1 数据库设计

本系统采用双存储架构：

MySQL 关系型数据库（库名：stain，字符集 utf8mb4）：存储检测任务元数据、图像记录、检测结果和区域明细，共 4 张数据表。

OSS 对象存储（MinIO 兼容）：存储用户上传的原始图像文件及检测结果图像文件，通过签名 URL 进行访问。

5.2 数据字典

5.2.1 检测任务表（inspection_tasks）

表描述：存储每次检测任务的主信息。

字段	类型	约束	说明
id	bigint unsigned	PK, AUTO_INCREMENT	任务 ID

字段	类型	约束	说明
user_id	varchar(64)	NOT NULL	用户 ID (来自 JWT)
building_name	varchar(255)	NOT NULL	建筑名称
location_floor	int	NULL	楼层号
location_section	varchar(255)	NULL	分区名称
description	text	NULL	描述信息
status	enum	NOT NULL, DEFAULT pending	状态 : pending/processing/done/failed
summary	text	NULL	检测结果摘要
stain_detected	tinyint(1)	NULL	是否检测到污渍 (0/1)
stain_type	varchar(255)	NULL	主要污渍类型
affected_area_percentage	decimal(6, 2)	NULL	污渍面积占比 (%)
error_message	text	NULL	错误信息
created_at	datetime	NOT NULL	创建时间
updated_at	datetime	NOT NULL	更新时间

索引:

```
idx_inspection_tasks_user_created (user_id, created_at DESC)
idx_inspection_tasks_user_status_created (user_id, status,
created_at DESC)
idx_inspection_tasks_status (status)
idx_inspection_tasks_building_name (building_name)
```

5.2.2 任务图像表 (task_images)

表描述: 存储检测任务关联的图像信息。

字段	类型	约束	说明
id	bigint unsigned	PK, AUTO_INCREMENT	记录 ID
task_id	bigint unsigned	FK -> inspection_tasks.id (CASCADE)	关联任务 ID
user_id	varchar(64)	NOT NULL	用户 ID
image_name	varchar(255)	NULL	图片名称
original_image_path	varchar(500)	NOT NULL	OSS 原始图像路径
processed_image_path	varchar(500)	NULL	OSS 检测后图像路

字段	类型	约束	说明
			径
mime_type	varchar(100)	NULL	MIME 类型
file_size	bigint unsigned	NULL	文件大小（字节）
created_at	datetime	NOT NULL	创建时间

5.2.3 检测结果表（detection_results）

表描述：存储检测任务的结果元数据和运行指标。

字段	类型	约束	说明
id	bigint unsigned	PK, AUTO_INCREMENT	记录 ID
task_id	bigint unsigned	FK, UNIQUE (CASCADE)	关联任务 ID（一对一）
user_id	varchar(64)	NOT NULL	用户 ID
status	enum	NOT NULL, DEFAULT pending	状态
summary	text	NULL	检测摘要
metrics	json	NULL	JSON 指标（runtimeMs, overallCleanliness, inferenceMode）
error_message	text	NULL	错误信息
created_at	datetime	NOT NULL	创建时间
updated_at	datetime	NOT NULL	更新时间
processed_at	datetime	NULL	处理完成时间

5.2.4 检测区域表（result_regions）

表描述：存储检测到的每个污渍区域的详细信息。

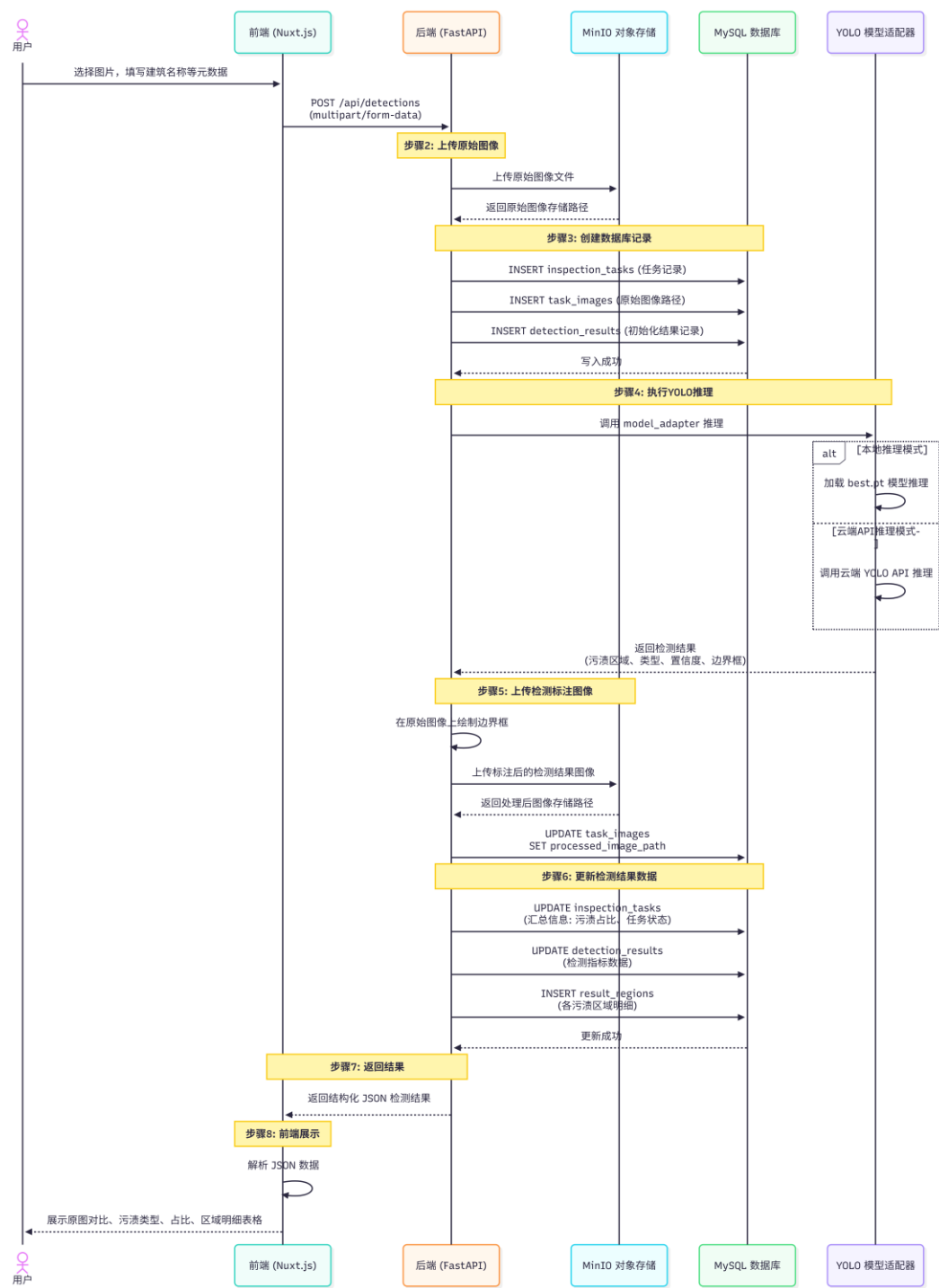
字段	类型	约束	说明
id	bigint unsigned	PK, AUTO_INCREMENT	记录 ID
task_id	bigint unsigned	FK -> inspection_tasks.id (CASCADE)	关联任务 ID
label	varchar(255)	NOT NULL	污渍类型：light/moderate/severe
confidence	decimal(5,4)	NOT NULL	置信度（0-1）

字段	类型	约束	说明
x1	decimal(8,5)	NOT NULL	边界框左上角 X（归一化坐标）
y1	decimal(8,5)	NOT NULL	边界框左上角 Y（归一化坐标）
x2	decimal(8,5)	NOT NULL	边界框右下角 X（归一化坐标）
y2	decimal(8,5)	NOT NULL	边界框右下角 Y（归一化坐标）
created_at	datetime	NOT NULL	创建时间

5.3 数据流图

5.3.1 污渍检测功能数据流图

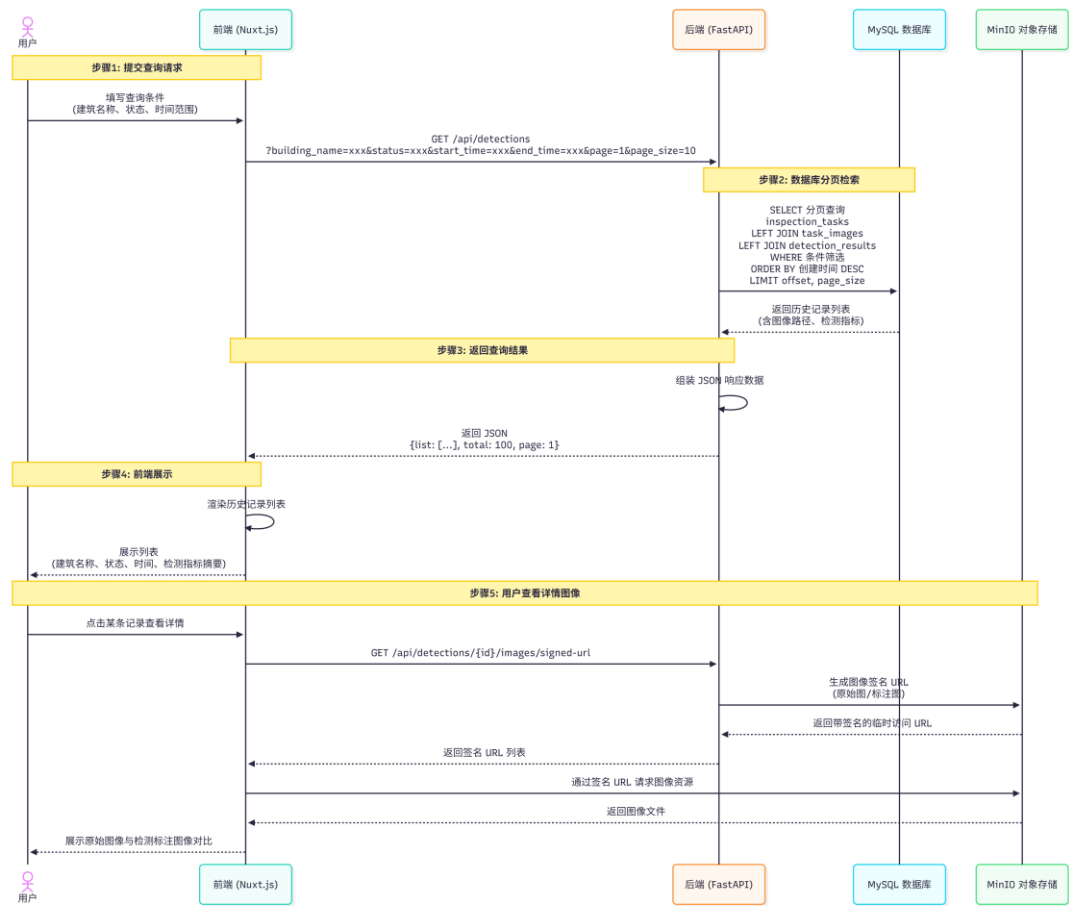
1. 用户通过前端选择图片，填写建筑名称等元数据，提交至后端（POST /api/detections, multipart/form-data）。
2. 后端接收图像文件，上传至 OSS 对象存储，获取原始图像存储路径。
3. 后 端 在 MySQL 中 创 建 inspection_tasks 、 task_images 、 detection_results 记录。
4. 后端调用 model_adapter 执行 YOLO 推理（本地 best.pt 或云端 API），获取检测结果（污渍区域、类型、置信度、边界框）。
5. 后 端 将 检 测 后 标 注 图 像 上 传 至 OSS ， 更 新 task_images.processed_image_path。
6. 后端更新 MySQL 中的检测结果（inspection_tasks 汇总、detection_results 指标、result_regions 区域明细）。
7. 后端将结构化 JSON 结果返回前端。
8. 前端解析 JSON 数据，展示检测结果（原图对比、污渍类型、占比、区域明细表格）。



5.3.2 历史记录查询数据流图

- 1. 用户在前端填写查询条件（建筑名称、状态、时间范围），提交查询请求至后端。
- 2. 后端根据查询条件从 MySQL 数据库中分页检索历史记录（inspection_tasks 关联 task_images、detection_results）。
- 3. 后端将查询结果（含图像路径、检测指标）以 JSON 格式返回前端。

4. 前端展示历史列表：用户点击详情时，通过 signed-url 接口获取图像签名 URL，从 OSS 加载并展示图像。



6. 组件设计

6.1 后端组件

6.1.1 应用入口（app/main.py）

FastAPI 应用入口，负责：创建 FastAPI 实例、配置 CORS 中间件（允许跨域请求）、注册 health 和 detections 两个 API 路由。

6.1.2 API 路由层（app/api/）

detections.py：定义检测相关的 7 个 RESTful API 端点，处理请求参数验证、文件格式校验，调用服务层执行业务逻辑。

health.py：提供健康检查端点，返回服务状态、时间戳和版本号。

6.1.3 核心层（app/core/）

config.py：基于 dataclass 的配置管理，从环境变量加载所有配置项（MySQL 连接、OSS 地址、JWT 密钥、YOLO 模型路径、云端模型配置等）。

auth.py: JWT 认证模块, 使用 PyJWT 库解码 HS256 Bearer Token, 从 payload 中提取 user_id (支持 sub/id/email 字段)。

response.py: 统一 JSON 响应格式封装 (ok/fail 辅助函数)。

6.1.4 服务层 (app/services/)

model_adapter.py: YOLO 模型适配器, 核心检测组件。

支持本地推理 (_detect_local) 和云端推理 (_detect_cloud) 两种模式

本地模式: 加载 best.pt 模型, 执行 predict, 解析 boxes 和 masks, 生成标注图像

云端模式: 通过 HTTP POST 调用远程推理 API, 解析返回的 JSON 结果, 使用 Pillow 绘制标注

污渍分类: light (轻度, 浅蓝)、moderate (中度, 青色)、severe (重度, 深蓝)

计算指标: affected_area_percentage (污渍占比)、overall_cleanliness (整体清洁度 = 100 - 污渍占比)

mysql_repo.py: MySQL 数据仓库, 封装所有数据库操作。

使用 PyMySQL 直连 MySQL, 每次操作独立创建连接 (无连接池)

封装 CRUD 操作: create_detection、update_detection_processing/done/failed、list_tasks、get_task_detail、delete_task 等

OSS 图像上传: upload_image、upload_processed_image (支持重试机制)

签名 URL 生成: create_signed_image_url

6.1.5 后台任务层 (app/workers/)

tasks.py: 后台异步检测任务。

process_detection_task: 接收图像字节数据, 执行同步推理 (用于 FastAPI BackgroundTasks)

process_detection_task_from_storage: 从 OSS 下载图像后执行推理 (用于重试场景)

6.2 前端组件

6.2.1 检测控制台 (pages/stonedirty/detection.vue)

新版检测页面, 采用左右分栏布局:

左侧: 上传图片卡片 (拖拽 / 点击上传) + 待检测队列面板 (PendingQueuePanelImproved 组件)

右侧: 检测结果展示卡片 (任务信息表格、区域明细表格、原图与检测后图片对比、PDF 导出)

支持批量上传、队列管理、单个/批量提交检测、轮询状态更新

6.2.2 历史记录 (pages/stonedirty/history.vue)

历史记录查询页面：

搜索栏：建筑名称输入、状态下拉筛选、时间范围选择器

操作按钮：查询、重置、刷新、导出 PDF、批量删除

数据表格：任务 ID、图片名称、建筑、状态、推理模式、总结、污渍占比、创建时间、操作列

详情抽屉：el-drawer 组件，展示完整检测信息、区域明细表格、原图与检测后图片

客户端缓存：15 分钟 Map 缓存，减少重复请求

6.2.3 API 封装层 (api/detections.ts)

封装所有与 FastAPI 后端的通信：

createDetectionTask: POST multipart/form-data 创建检测任务

getDetectionTask: GET 获取任务详情

getDetectionList: GET 分页查询列表

getDetectionSignedUrl: GET 获取图像签名 URL

retryDetection: POST 重试失败任务

deleteDetection: DELETE 删除单条记录

batchDeleteDetections: DELETE 批量删除

所有请求自动携带 localStorage 中的 authToken 作为 Bearer Token

6.3 组件依赖关系

后端组件依赖关系：

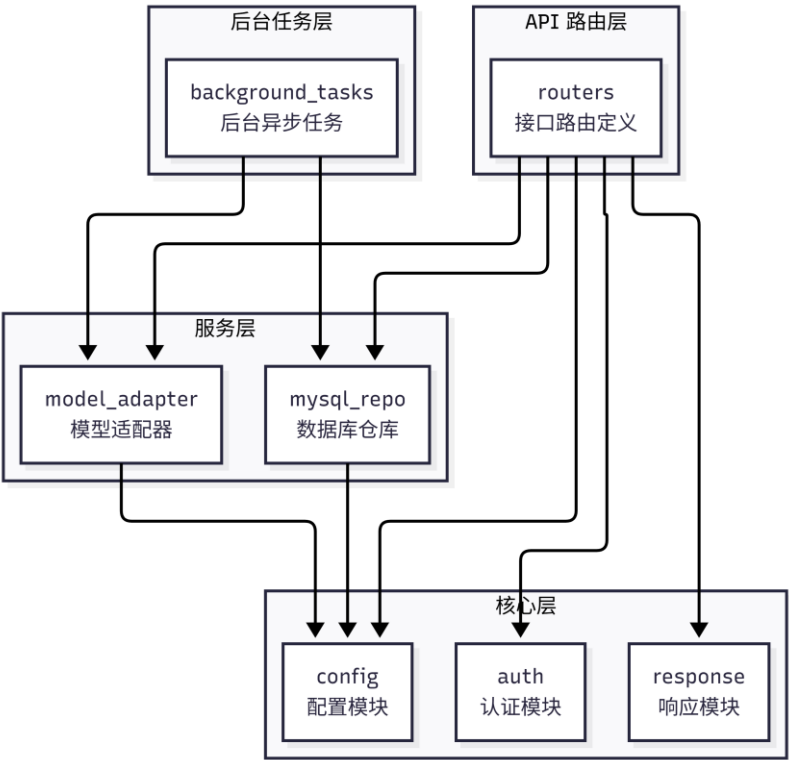
API 路由层 → 服务层 (model_adapter, mysql_repo)

API 路由层 → 核心层 (auth, config, response)

服务层 (mysql_repo) → 核心层 (config)

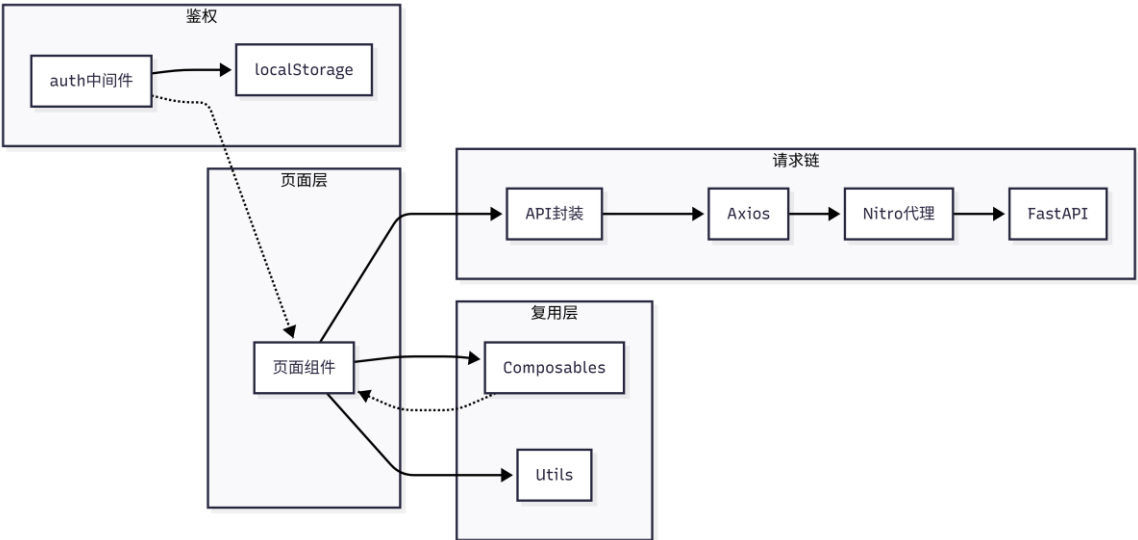
服务层 (model_adapter) → 核心层 (config)

后台任务层 → 服务层 (model_adapter, mysql_repo)



前端组件依赖关系:

页面组件 → API 封装层 → Axios → Nitro 代理 → FastAPI 后端
页面组件 → Composables (usePendingQueue) → 响应式状态
页面组件 → Utils (detectionPdfExport, compressImage)
全局中间件 (auth.global.js) → localStorage 中的 authToken



7. 用户界面设计

本系统前端基于 Nuxt.js 3 框架开发，遵循简洁直观的设计原则，确保用户可高效完成图像上传、污渍检测、结果查看、报告下载及历史查询等核心操作流程。污渍检测模块作为子模块集成于智慧幕墙管理平台，通过侧边栏导航进入。

7.1 系统整体布局

系统采用侧边栏 + 主内容区的经典后台管理布局：

侧边栏：包含所有功能模块的导航入口（振动监测、玻璃裂纹检测、玻璃平整度检测、性能评估、污渍检测、腐蚀检测等）

主内容区：根据当前路由渲染对应的页面组件

全局认证：未登录用户自动跳转至登录页面；权限控制基于 localStorage 中的权限字段（access_system_a ~ access_system_z, is_superuser）

7.2 检测控制台页面

检测控制台（/stonedirty/detection）是污渍检测的核心操作页面，采用左右分栏布局。

7.2.1 左侧操作区

（1）上传图片卡片：

支持拖拽上传和点击上传两种方式

支持同时选择多张图片批量上传

支持 JPEG、PNG 格式，单文件不超过 50MB

超过 100KB 的图片自动压缩后上传

（2）待检测队列面板：

显示所有已上传待检测的图片列表

每项显示：图片预览缩略图、自定义名称、上传状态、进度条

支持操作：单独上传、删除、切换推理模式（本地模型/云端模型）

批量操作：一键上传全部、清除已完成项

上传完成后可直接点击查看检测结果

7.2.2 右侧展示区

检测结果卡片：

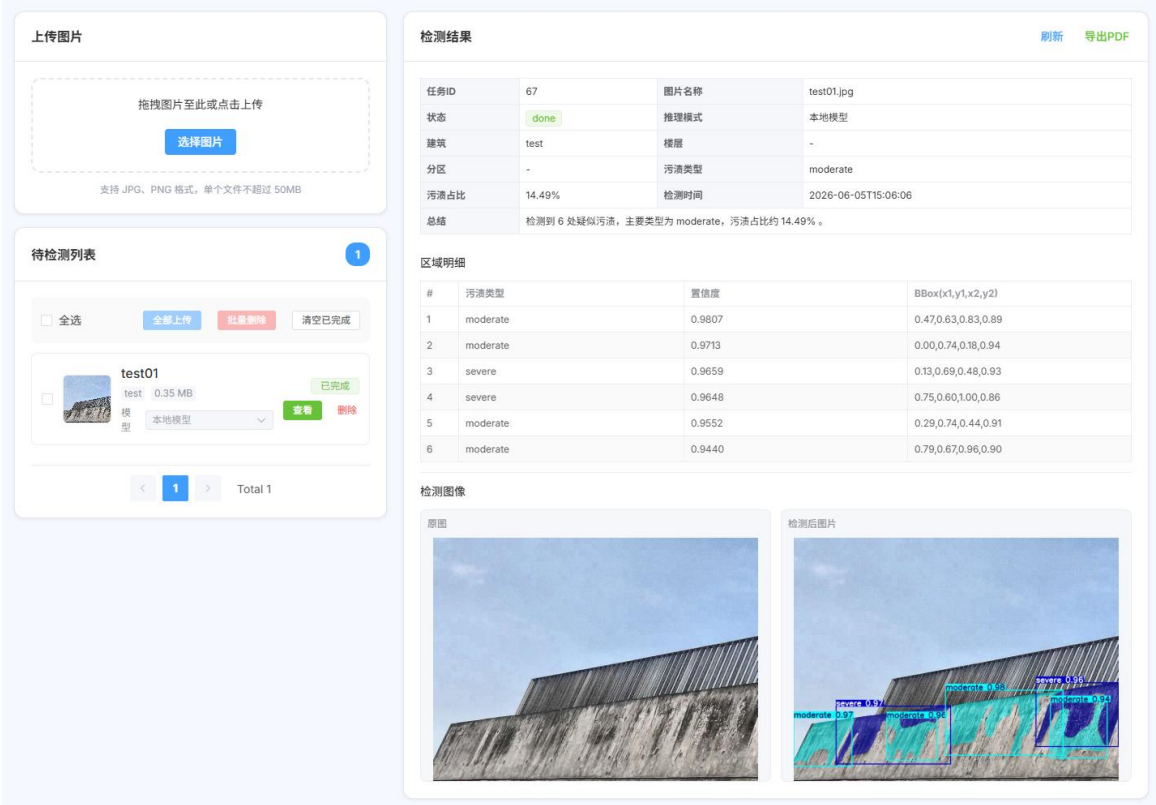
任务信息表格：任务 ID、图片名称、状态（彩色标签）、推理模式、建筑、楼层、分区、污渍类型、污渍占比、检测时间、总结

区域明细表格：每个检测到的污渍区域的类型、置信度（4 位小数）、边界框坐标 (x1, y1, x2, y2)

错误提示：失败时显示错误信息 Alert

图像对比展示：原图与检测后图片并排显示，支持点击预览大图

操作按钮：刷新（重新获取签名 URL）、导出 PDF



7.3 历史记录页面

历史记录页面（/stonedirty/history）提供检测历史的查询、浏览和管理功能。

7.3.1 搜索与筛选

建筑名称：文本输入，支持模糊搜索

状态下拉：全部/待处理/处理中/已完成/失败

时间范围：日期时间范围选择器

操作按钮：查询、重置、刷新、导出 PDF（批量）、删除（批量）

7.3.2 数据表格

支持多选（checkbox 列）

显示字段：任务 ID、图片名称、建筑、状态（彩色标签）、推理模式、总结、污渍占比、创建时间

操作列：详情、导出 PDF、重试（仅失败状态）、删除

分页：底部 Pagination 组件，显示总数

7.3.3 详情抽屉

从右侧滑出的 el-drawer 组件，宽度 50%

任务信息描述列表（el-descriptions）

区域明细表格

错误信息 Alert（失败时）
原图与检测后图片对比展示
定时刷新签名 URL（每 1 小时）

检测历史

返回检测工作台

建筑 状态全部 时间范围开始时间至结束时间 查询 重置 刷新 导出PDF 删除

☐	任务ID	图片名称	建筑	状态	推理模式	总结	污渍占比	创建时间	操作
☐	67	test01.jpg	test	done	本地模型	检测到 6 处疑似污渍，主要类型为 moderat...	14.49%	2026-06-05T15:06:06	详情 导出PDF 删除
☐	66	屏幕截图 2026-06-04 203840.png	屏幕截图	done	本地模型	检测到 4 处疑似污渍，主要类型为 severe...	5.36%	2026-06-04T12:43:31	详情 导出PDF 删除
☐	64	test02.jpg	test	done	本地模型	检测到 13 处疑似污渍，主要类型为 severe...	19.02%	2026-05-18T07:03:14	详情 导出PDF 删除
☐	63	test06.jpg	test	done	本地模型	检测到 8 处疑似污渍，主要类型为 light, 污...	6.37%	2026-05-18T06:58:20	详情 导出PDF 删除
☐	62	test04.jpg	test	done	本地模型	检测到 4 处疑似污渍，主要类型为 moderat...	1.79%	2026-05-18T06:57:34	详情 导出PDF 删除
☐	61	test05.jpg	test	done	本地模型	检测到 8 处疑似污渍，主要类型为 moderat...	22.26%	2026-05-18T06:56:48	详情 导出PDF 删除
☐	60	test03.jpg	test	done	本地模型	检测到 3 处疑似污渍，主要类型为 light, 污...	31.57%	2026-05-18T06:56:19	详情 导出PDF 删除
☐	59	test02.jpg	test	done	本地模型	检测到 13 处疑似污渍，主要类型为 severe...	19.02%	2026-05-18T06:55:49	详情 导出PDF 删除
☐	58	test07.jpg	test	done	本地模型	检测到 5 处疑似污渍，主要类型为 moderat...	29.05%	2026-05-18T06:54:24	详情 导出PDF 删除

检测详情

任务ID	67	图片名称	test01.jpg
状态	done	推理模式	本地模型
建筑	test	楼层	-
分区	-	污渍类型	moderate
污渍占比	14.49%	检测时间	2026-06-05T15:06:06
总结	检测到 6 处疑似污渍，主要类型为 moderate，污渍占比约 14.49%。		

#	污渍类型	置信度	BBox(x1,y1,x2,y2)
1	moderate	0.9807	0.47,0.63,0.83,0.89
2	moderate	0.9713	0.00,0.74,0.18,0.94
3	severe	0.9659	0.13,0.69,0.48,0.93
4	severe	0.9648	0.75,0.60,1.00,0.86
5	moderate	0.9552	0.29,0.74,0.44,0.91
6	moderate	0.9440	0.79,0.67,0.96,0.90

原图

检测后图片

7.4 检测报告导出

系统支持 PDF 格式的检测报告导出，使用 jsPDF + html2canvas 在客户端生成 PDF 文件。

- 检测控制台页面：导出当前查看的单条检测结果报告
- 历史记录页面：导出单条历史记录报告，或批量导出勾选的多条记录报告
- 报告内容包括：任务信息、建筑信息、污渍类型、污渍占比、区域明细、原图与检测后图片等

幕墙污渍检测分析报告

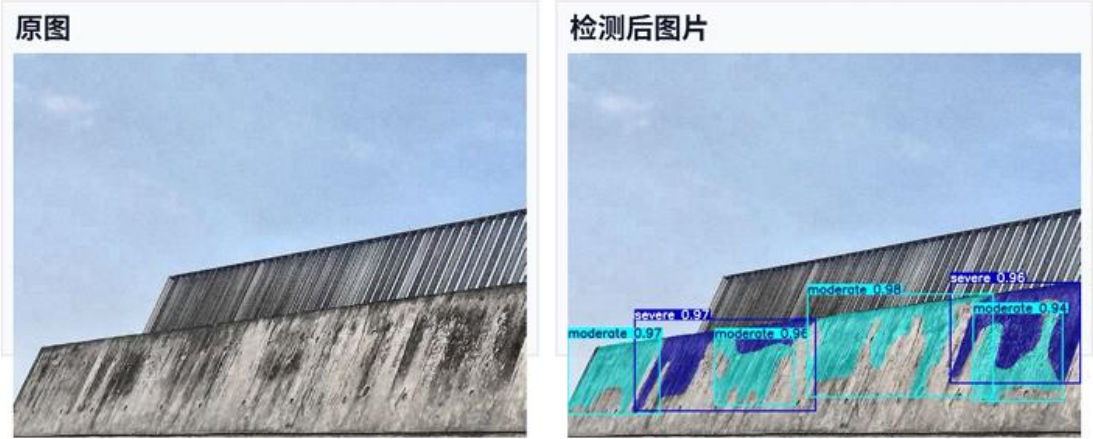
生成时间：2026-06-06 18:24:27

任务ID：67
图片名称：test01.jpg
建筑：test
楼层：-
分区：-
状态：done
推理模式：本地模型
污渍类型：moderate
污渍占比：14.49%
总结：检测到 6 处疑似污渍，主要类型为 moderate，污渍占比约 14.49%。
检测时间：2026-06-05 15:06:06

区域明细

#	Label	Confidence	BBox(x1,y1,x2,y2)
1	moderate	0.9807	0.47,0.63,0.83,0.89
2	moderate	0.9713	0.00,0.74,0.18,0.94
3	severe	0.9659	0.13,0.69,0.48,0.93
4	severe	0.9648	0.75,0.60,1.00,0.86
5	moderate	0.9552	0.29,0.74,0.44,0.91
6	moderate	0.9440	0.79,0.67,0.96,0.90

检测图像



8. 安全设计

8.1 身份认证

系统采用 JWT (JSON Web Token) Bearer Token 认证机制:

Token 由独立的用户认证服务 (8.159.143.133:8000) 签发, 使用 HS256 算法签名

前端将 Token 存储于 localStorage (键名: authToken)

所有 API 请求通过 Axios 拦截器自动携带 Authorization: Bearer <token> 请求头

后端通过 PyJWT 解码 Token, 从 payload 中提取 user_id (支持 sub、id、email 字段)

Token 无效或过期时返回 401 Unauthorized, 前端自动跳转登录页

8.2 权限控制

前端通过全局路由中间件 (middleware/auth.global.js) 实现基于权限的访问控制:

权限字段存储于 Token payload 中: access_system_a ~ access_system_z 和 is_superuser

每个功能模块映射到特定的权限字段, 如污渍检测模块需要 access_system_f 权限

未授权用户访问受限页面时自动跳转至首页

超级管理员 (is_superuser=true) 可访问所有模块

9. 部署设计

9.1 后端部署

后端采用 Docker 容器化部署:

基础镜像: python:3.12-slim

系统依赖: 安装 libgl1-mesa-glx 等 OpenCV/YOLO 所需的图形库

Python 依赖: 通过 requirements.txt 安装 (使用阿里云 PyPI 镜像加速)

暴露端口: 8081

启动命令: uvicorn app.main:app --host 0.0.0.0 --port 8081

Docker Compose 配置:

服务名: backend, 容器名: stain-backend

端口映射: 8081:8081

卷挂载: ./models (YOLO 模型文件)、.env (环境变量)、./app (应用代码)

重启策略: unless-stopped

9.2 前端部署

前端采用 Nuxt.js 3 内置服务器部署:

开发模式: pnpm dev (默认端口 3000)

生产构建: nuxt build + node .output/server/index.mjs

静态生成: nuxt generate (适用于纯静态部署)

CSR 模式: ssr: false, 所有渲染在客户端完成

反向代理: 生产环境建议通过 Nginx 反向代理, 配置 300 秒超时 (大图像处理场景)

9.3 环境变量配置

后端环境变量 (通过 .env 文件或 Docker 环境变量注入):

变量名	默认值	说明
APP_NAME	Stone Curtain Wall Stain Detection API	应用名称
APP_ENV	development	运行环境
CORS_ORIGINS	http://localhost:80	允许的跨域来源
OSS_BASE_URL	http://8.159.143.133:9000	OSS 服务地址
OSS_UPLOAD_ENDPOINT	/oss/upload/output	OSS 上传端点
OSS_USERNAME	stain-detection	OSS 用户名
OSS_PASSWORD	tongji-icw-3567	OSS 密码
MYSQL_HOST	8.159.143.133	MySQL 主机
MYSQL_PORT	3306	MySQL 端口
MYSQL_USER	stain	MySQL 用户名
MYSQL_PASSWORD	stain	MySQL 密码
MYSQL_DATABASE	stain	MySQL 数据库名
JWT_SECRET		JWT 签名密钥
SYNC_SIZE_THRESHOLD_BYTES	2097152	同步处理阈值 (2MB)
YOLO_MODEL_PATH	models/best.pt	YOLO 模型文件路径
YOLO_CONFIDENCE_THRESHOLD	0.25	YOLO 置信度阈值
CLOUD_MODEL_URL		云端推理 API 地址
CLOUD_MODEL_API_KEY		云端推理 API 密钥

变量名	默认值	说明
CLOUD_MODEL_IOU_THRESHOLD	0.7	云端模型 IoU 阈值
CLOUD_MODEL_IMGSZ	640	云端模型输入图像尺寸
CLOUD_MODEL_TIMEOUT_SECONDS	60	云端推理超时时间

10. 附录

10.1 缩略语与术语表

缩略语/术语	全称	说明
YOLO	You Only Look Once	实时目标检测深度学习模型系列
FastAPI	—	Python 异步 Web 框架
Nuxt.js	—	基于 Vue.js 的全栈框架
OSS	Object Storage Service	对象存储服务
MinIO	—	开源对象存储服务（兼容 S3 协议）
JWT	JSON Web Token	JSON 网络令牌，用于身份认证
CSR	Client-Side Rendering	客户端渲染模式
SSR	Server-Side Rendering	服务端渲染模式
API	Application Programming Interface	应用程序编程接口
RESTful	Representational State Transfer	表述性状态转移架构风格
Pydantic	—	Python 数据验证库
PyJWT	—	Python JWT 编解码库
PyMySQL	—	Python MySQL 数据库驱动
Pillow (PIL)	Python Imaging Library	Python 图像处理库
IoU	Intersection over Union	交并比，目标检测评估指标
BBBox	Bounding Box	边界框，目标检测中的检测框

10.2 参考文档

10.2.1 项目文档

SRS_石材幕墙污渍检测系统.docx（软件需求规格说明）

10.2.2 技术参考

YOLOv8 官方文档: <https://docs.ultralytics.com/>

FastAPI 官方文档: <https://fastapi.tiangolo.com/>

Nuxt.js 3 官方文档: <https://nuxt.com/>

Vue 3 官方文档: <https://vuejs.org/>

Element Plus 官方文档: <https://element-plus.org/>

Docker 官方文档: <https://docs.docker.com/>

MySQL 8.0 参考手册: <https://dev.mysql.com/doc/>

IEEE 1016-2009 软件设计规格说明标准